

# ■ Hoja de Referencia Rápida: Spring Boot 3.x (Auto-Config & REST APIs)

---

## ■ 1. Anotaciones Principales de Arranque y Configuración

| Anotación                             | Propósito y Función                                                                                                   |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>@SpringBootApplication</code>   | Combina <code>@SpringBootConfiguration</code> , <code>@EnableAutoConfiguration</code> y <code>@ComponentScan</code> . |
| <code>@EnableAutoConfiguration</code> | Activa autoconfiguración según los Starters del <code>pom.xml</code> .                                                |
| <code>@ComponentScan</code>           | Escanea paquetes buscando anotaciones <code>@Component</code> .                                                       |
| <code>@ConfigurationProperties</code> | Mapea estructuras del <code>application.yml</code> a POJOs.                                                           |
| <code>@EnableScheduling</code>        | Habilita tareas programadas ( <code>@Scheduled</code> ).                                                              |
| <code>@EnableAsync</code>             | Habilita ejecución asíncrona ( <code>@Async</code> ).                                                                 |

## ■ 2. Anotaciones para Controladores RESTful (`Spring Web MVC`)

---

```

@RestController
@RequestMapping("/api/v1/productos")
@CrossOrigin(origins = "*")
public class ProductoController {

    private final ProductoService productoService;

    public ProductoController(ProductoService service) {
        this.productoService = service;
    }

    // GET /api/v1/productos?cat=electronica
    @GetMapping
    public ResponseEntity<List<ProductoDTO>> listar(
        @RequestParam(required = false) String cat) {
        return ResponseEntity.ok(
            productoService.listarPorCategoria(cat)
        );
    }

    // GET /api/v1/productos/105
    @GetMapping("/{id}")
    public ResponseEntity<ProductoDTO> obtenerPorId(
        @PathVariable("id") Long id) {
        return ResponseEntity.ok(
            productoService.buscarPorId(id)
        );
    }

    // POST /api/v1/productos
    @PostMapping
    public ResponseEntity<ProductoDTO> crear(
        @Valid @RequestBody ProductoDTO dto) {
        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(productoService.guardar(dto));
    }

    // PUT /api/v1/productos/105
    @PutMapping("/{id}")
    public ResponseEntity<ProductoDTO> actualizar(
        @PathVariable Long id,
        @Valid @RequestBody ProductoDTO dto) {
        return ResponseEntity.ok(
            productoService.actualizar(id, dto)
        );
    }
}

```

```

    }

    // DELETE /api/v1/productos/105
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id) {
        productoService.eliminar(id);
        return ResponseEntity.noContent().build();
    }
}

```

### ■ ■ 3. Manejo Centralizado de Excepciones HTTP (RFC 7807)

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RecursoNoEncontradoException.class)
    public ProblemDetail manejarNoEncontrado(
        RecursoNoEncontradoException ex) {
        ProblemDetail problem = ProblemDetail
            .forStatusAndDetail(
                HttpStatus.NOT_FOUND, ex.getMessage()
            );
        problem.setTitle("Recurso No Encontrado");
        problem.setProperty("timestamp", Instant.now());
        return problem;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail manejarValidacion(
        MethodArgumentNotValidException ex) {
        ProblemDetail problem = ProblemDetail
            .forStatusAndDetail(
                HttpStatus.BAD_REQUEST, "Error de validacion"
            );
        problem.setTitle("Solicitud Invalida");

        Map<String, String> errores = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(e ->
            errores.put(e.getField(), e.getDefaultMessage()));

        problem.setProperty("invalidFields", errores);
        return problem;
    }
}

```

```
}  
}
```

---

## ■ 4. Mapeo Tipado de Configuración (@ConfigurationProperties)

```
@Configuration  
@ConfigurationProperties(prefix = "app.jwt")  
public class JwtProperties {  
    private String secret;  
    private long expirationMs;  
  
    public String getSecret() {  
        return secret;  
    }  
    public void setSecret(String secret) {  
        this.secret = secret;  
    }  
    public long getExpirationMs() {  
        return expirationMs;  
    }  
    public void setExpirationMs(long expirationMs) {  
        this.expirationMs = expirationMs;  
    }  
}
```

---

## ■ 5. Endpoints Clave de Spring Boot Actuator

| Endpoint                       | Utilidad                                          |
|--------------------------------|---------------------------------------------------|
| <code>/actuator/health</code>  | Salud de la app (UP/DOWN), base de datos y disco. |
| <code>/actuator/metrics</code> | Métricas de JVM, CPU y peticiones HTTP.           |

|                        |                                      |
|------------------------|--------------------------------------|
| <b>/actuator/env</b>   | Propiedades de entorno configuradas. |
| <b>/actuator/beans</b> | Lista completa de Beans registrados. |